

AI Business Assistant

Atelier Prompt Engineering — Rapport de réalisation

Sommaire

#	Partie
1	Anatomie d'un prompt
2	Comparer les techniques de prompting
3	Prompt Engineering et raisonnement
4	Sorties structurées
5	Prompts pour les applications métier
6	Prompt Engineering pour le Machine Learning
7	Prompt Engineering et RAG
8	Évaluation et optimisation des prompts
9	Bonus

Partie 1 — Anatomie d'un prompt

Consigne

Construire un prompt, à partir de ses différentes composantes, pour résoudre un problème comme « Je souhaite analyser les retours de clients d'une entreprise. »

Éléments attendus

- Identifier les composantes du prompt (rôle, contexte, tâche, contraintes, format de sortie)
- Rédiger le prompt final

Prompt

«

Rôle

Tu es un analyste spécialisé dans l'étude de la satisfaction client pour une entreprise de services.

Contexte

L'entreprise reçoit régulièrement des retours clients (avis, commentaires, réponses à des enquêtes) via différents canaux. Ces retours ne sont ni triés ni analysés systématiquement.

Tâche

Analyse les retours clients suivants et identifie : (1) le sentiment général de chaque retour, (2) les thèmes récurrents évoqués (ex : livraison, service client, qualité du produit, prix), (3) les problèmes les plus critiques à traiter en priorité.

Contraintes

- Ne pas inventer d'informations absentes des retours.
- Rester factuel et objectif, sans exagérer un problème mineur.
- Si un retour est ambigu ou incomplet, le signaler plutôt que de forcer une interprétation.

Format de sortie

Présente le résultat sous forme de liste structurée : un sentiment (positif/négatif/neutre) et un ou plusieurs thèmes par retour, suivis d'une synthèse globale des priorités. »

Partie 2 — Comparer les techniques de prompting

Consigne

Tester puis évaluer les résultats obtenus avec les techniques zero-shot, one-shot, few-shot et un prompt structuré sur une demande de classification de commentaire (positif, négatif, neutre) : « Le service est rapide mais l'application plante régulièrement. »

Éléments attendus

- Prompt et résultat pour chaque technique (zero-shot, one-shot, few-shot, structuré)
- Comparaison / analyse des résultats obtenus

Prompt(s) et résultat(s)

Zero-shot :

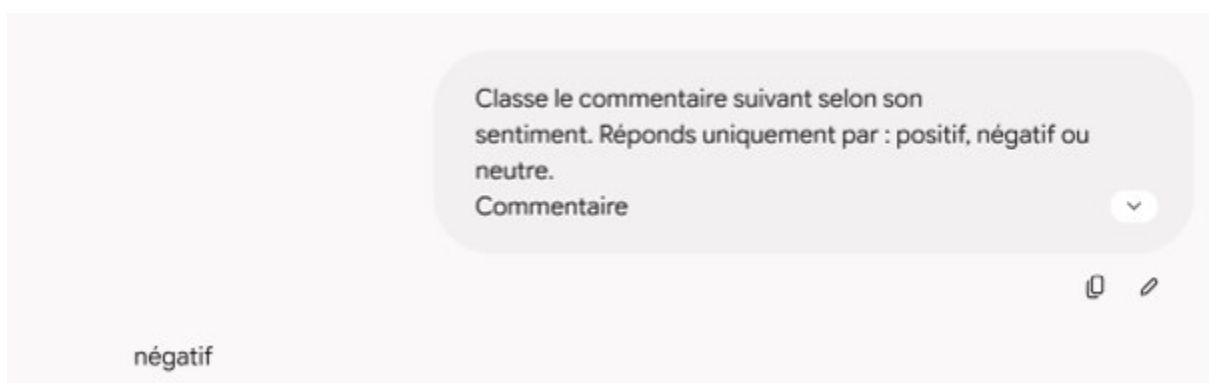
« Classe le commentaire suivant selon son sentiment. Réponds uniquement par : positif, négatif ou neutre.

Commentaire

: "Le service est rapide mais l'application plante régulièrement."

Classes possibles :

positif, négatif, neutre. »



One-shot :

« Voici un exemple de classification de commentaire selon son sentiment :

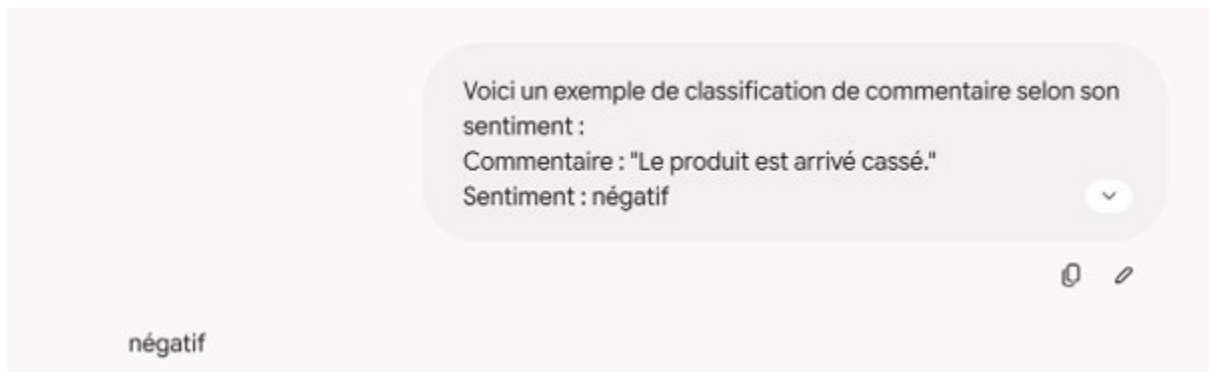
Commentaire : "Le produit est arrivé cassé."

Sentiment : négatif

Classe maintenant le commentaire suivant de la même manière :

Commentaire : "Le service est rapide mais l'application plante régulièrement."

Sentiment : »



Few-shot :

« Voici des exemples de classification de commentaires selon leur sentiment :

Commentaire : "Le produit est arrivé cassé"

Sentiment : négatif

Commentaire : "Livraison rapide et produit conforme, je recommande."

Sentiment : positif

Commentaire : "L'application fonctionne, rien de particulier à signaler."

Sentiment : neutre

Commentaire : "Le prix est correct mais le service client met du temps à répondre."

Sentiment : négatif

Classe maintenant le commentaire suivant de la même manière :

Commentaire : "Le service est rapide mais l'application plante régulièrement."

Sentiment : »

Voici des exemples de classification de commentaires selon leur sentiment :

Commentaire : "Le produit est arrivé cassé."

Sentiment : négatif



négatif

Structured Prompting :

« Tu es un analyste chargé de classer des commentaires clients selon leur sentiment pour aider une entreprise à prioriser ses actions.

Tâche : classe le commentaire suivant dans l'une des classes : positif, négatif, neutre.

Contraintes :

- Base ta décision uniquement sur le contenu du commentaire, sans supposer d'informations absentes.

- Si le commentaire contient à la fois des aspects positifs et négatifs, choisis la classe qui reflète l'impression dominante ou choisis "neutre" si aucun aspect ne domine clairement.

Format de sortie : réponds uniquement au format JSON suivant, sans texte additionnel :

```
{"sentiment": "positif" | "négatif" | "neutre"}
```

Commentaire : "Le service est rapide mais l'application plante régulièrement." »

Tu es un analyste chargé de classer des commentaires clients selon leur sentiment pour aider une entreprise à prioriser ses actions.

Tâche : classe le commentaire suivant dans l'une des class 

```
{"sentiment": "neutre"}
```

Analyse / remarques :

Sur un commentaire volontairement ambigu ("rapide" mais "plante régulièrement"), zero-shot, one-shot et few-shot donnent tous **négatif**, le modèle privilégiant par défaut le problème récurrent sur l'avantage ponctuel. Le few-shot n'a pas changé ce résultat car ses exemples couvraient des cas déjà tranchés, pas des cas mixtes. Le prompt structuré, lui, donne **neutre**, directement grâce à la règle de décision explicite qu'il contient ("neutre si aucun aspect ne domine clairement"). Cela montre que plus d'exemples améliore surtout le format de sortie, tandis qu'une règle de décision explicite améliore la logique de raisonnement sur les cas ambigus. Pour une application métier fiable, le prompt structuré est donc à privilégier dès que la tâche comporte un risque d'ambiguïté.

Partie 3 — Prompt Engineering et raisonnement

Éléments attendus

- Décomposition du prompt initial
- Prompt d'analyse + résultat
- Prompt de vérification + résultat

Consigne

- 1) Décomposer le prompt « Analyse ces avis clients et donne-moi les problèmes les plus importants ainsi que les recommandations. »

le prompt mélange deux tâches distinctes (extraction de problèmes + génération de recommandations) sans les séparer, ne définit pas de critère de priorisation, et ne pose aucune contrainte contre l'invention d'information — ce qui est risqué pour une tâche d'analyse censée rester factuelle.

- 2) Proposer un texte à analyser avec contraintes, puis un second prompt de vérification (informations non justifiées, contradictions, informations absentes, hallucinations, respect des contraintes).

- a. Texte d'avis clients (exemple à utiliser)

Avis 1 : "La livraison a pris 8 jours au lieu de 3 annoncés, très décevant."

Avis 2 : "Le support technique ne répond jamais avant 48h, c'est trop long."

Avis 3 : "Le produit fonctionne bien mais l'emballage est arrivé abîmé."

Avis 4 : "Application facile à utiliser, rien à redire."

Avis 5 : "Encore un retard de livraison ce mois-ci, ça devient récurrent."

- b. Premier prompt (avec contraintes)

Tu es un analyste chargé d'exploiter des avis clients pour une entreprise de e-commerce.

Tâche : analyse les avis clients fournis ci-dessous et identifie les 3 problèmes les plus importants, en te basant sur leur fréquence d'apparition et leur impact perçu sur l'expérience client. Pour chaque problème, propose une recommandation concrète.

Contraintes :

- *N'invente aucune information absente des avis.*
- *Base uniquement ton analyse sur le contenu fourni.*
- *Si un avis est positif ou ne signale pas de problème, ne le fais pas apparaître comme un problème.*
- *Indique, pour chaque problème identifié, sur quel(s) avis il s'appuie (numéro de l'avis).*

Format de sortie : liste structurée "Problème / Avis concernés/Recommandation".

Avis clients : (les avis clients) »

Voici l'analyse des avis clients :

1. **Problème : Retards de livraison répétés et non-respect des délais affichés**
 - **Avis concernés :** Avis 1, Avis 5
 - **Recommandation :** Réévaluer les engagements de délais avec les transporteurs, auditer la chaîne logistique pour réduire les retards et ajuster les délais estimatifs affichés sur le site pour qu'ils soient plus réalistes.
2. **Problème : Délais de réponse trop longs du support technique**
 - **Avis concernés :** Avis 2
 - **Recommandation :** Augmenter les effectifs du support technique aux heures de pointe ou mettre en place des réponses automatisées (type FAQ dynamique ou système d'enregistrement de tickets) pour accuser réception et orienter le client plus rapidement.
3. **Problème : Emballage dégradé lors du transport**

c. Second prompt

Voici ta réponse précédente à la demande d'analyse des avis clients :

(reponse du 1^e prompt)

Vérifie rigoureusement cette réponse et signale, pour chacun des points suivants, s'il y a un problème détecté (avec justification) ou non :

- 1. Informations non justifiées : chaque problème et chaque recommandation s'appuie-t-il bien sur un avis cité ?*
- 2. Contradictions : y a-t-il des incohérences entre les problèmes identifiés et les avis d'origine ?*
- 3. Informations absentes : un problème important présent dans les avis a-t-il été oublié ?*
- 4. Hallucinations : la réponse mentionne-t-elle un fait, un chiffre ou un détail qui n'apparaît dans aucun avis ?*

5. Respect des contraintes : le format demandé (liste structurée avec avis cités) a-t-il été respecté ?

Conclus par un verdict global : réponse fiable, à corriger partiellement, ou à refaire — avec justification.

Verdict global

Réponse fiable

La réponse originale répond avec précision et rigueur à la demande. Les 3 problèmes majeurs ont été correctement extraits, regroupés et hiérarchisés, les avis sources sont fidèlement rattachés et les contraintes de format sont entièrement respectées. Les pistes d'action suggérées apportent la valeur managériale demandée tout en restant directement alignées sur la nature des dysfonctionnements rapportés.

Partie 4 — Sorties structurées

Consigne

À partir d'une réponse libre du LLM, créer un prompt demandant une sortie JSON avec les champs sentiment, categorie, urgence, probleme, confiance (types et valeurs autorisées précisés), puis définir des règles de validation (JSON valide, pas de propriété supplémentaire, valeurs autorisées, confiance entre 0 et 1).

Éléments attendus

- Prompt demandant la sortie JSON
- Exemple de JSON obtenu
- Règles de validation définies

Prompt(s) et résultat(s)

1.

Tu es un système d'analyse de commentaires clients pour une application métier.

Tâche : analyse le commentaire suivant et retourne le résultat au format JSON avec exactement les champs suivants :

- *"sentiment" (chaîne de caractères) : "positif", "negatif" ou "neutre"*
- *"categorie" (chaîne de caractères) : le thème principal du commentaire (ex : "livraison", "service client", "produit", "prix", "application")*
- *"urgence" (chaîne de caractères) : "faible", "moyenne" ou "élevée"*
- *"probleme" (chaîne de caractères) : description courte du problème signalé, ou "aucun" si le commentaire n'en signale pas*
- *"confiance" (nombre décimal entre 0 et 1) : ton niveau de confiance dans cette classification*

Contraintes :

- *Réponds uniquement avec le JSON, sans texte avant ou après.*
- *N'invente aucune information absente du commentaire.*

Commentaire : "Le service est rapide mais l'application plante régulièrement."

```
{
  "sentiment": "neutre",
  "categorie": "application",
  "urgence": "moyenne",
  "probleme": "l'application plante régulièrement",
  "confiance": 0.9
}
```

2. Tu es un système d'analyse de commentaires clients pour une application métier.

Tâche : analyse le commentaire suivant et retourne un JSON avec les champs "sentiment", "categorie", "urgence", "probleme", "confiance".

Règles de validation strictes à respecter :

1. Le JSON doit être syntaxiquement valide (guillemets doubles, pas de virgule finale).
2. Aucune propriété supplémentaire ne doit être ajoutée en dehors des 5 champs demandés.
3. "sentiment" doit être exactement l'une des valeurs : "positif", "negatif", "neutre" (pas de variante, pas de majuscule).
4. "urgence" doit être exactement l'une des valeurs : "faible", "moyenne", "élevée".
5. "confiance" doit être un nombre décimal strictement compris entre 0 et 1 (inclus).
6. Si une information ne peut pas être déterminée à partir du commentaire, indique la valeur la plus prudente possible plutôt que d'inventer (ex : "categorie": "autre").
7. Ne retourne rien d'autre que le JSON : pas d'explication, pas de balises markdown ``json.

Commentaire : "Le service est rapide mais l'application plante régulièrement."

```
{  
  "sentiment": "neutre",  
  "categorie": "application",  
  "urgence": "moyenne",  
  "probleme": "l'application plante régulièrement",  
  "confiance": 0.9  
}
```

Partie 5 — Prompts pour les applications métier

Consigne

Créer 5 prompts métier :

- (1) résumé de document ≤ 250 mots sans invention,
- (2) traduction FR→EN fidèle,
- (3) classification de ticket informatique en JSON (categorie, justification),
- (4) extraction de données de facture en JSON (numéro_facture, date, client, montant_ht, tva, montant_ttc),
- (5) rédaction d'un email client pour un retard de livraison (ton professionnel, ≤ 150 mots).

Éléments attendus

- Les 5 prompts rédigés
- Un exemple de résultat pour chacun

Prompt(s) et résultat(s)

Prompt 1 :

Tu es un assistant chargé de résumer des documents professionnels pour des collaborateurs pressés.

Tâche : résume le document suivant en respectant les contraintes ci-dessous.

Contraintes :

- *Maximum 250 mots.*
- *Conserve toutes les informations factuelles importantes (chiffres, dates, noms).*
- *Identifie clairement les objectifs du document.*
- *Identifie clairement les résultats obtenus.*
- *Identifie clairement les recommandations formulées.*
- *N'invente aucune information absente du document.*

Document : « Doc_resume.pdf »

Voici le résumé du document :

Objectifs du projet

Lancé en janvier 2025, le projet « ÉcoTri 2026 » de la municipalité d'ÉcoVille vise deux objectifs d'ici la fin de l'année : [PDF](#)

- Réduire de 25 % le volume de déchets ménagers non recyclés. [PDF](#)
- Augmenter de 15 points le taux de satisfaction des usagers concernant la collecte. [PDF](#)

Résultats obtenus (au 30 juin 2026)

- **Réduction des déchets** : Diminution de 18 % des déchets non recyclés (en dessous de l'objectif initial). [PDF](#)
- **Satisfaction des usagers** : Hausse de 22 points du taux de satisfaction (dépassant l'objectif), soutenue par l'installation de 140 nouveaux conteneurs connectés. [PDF](#)
- **Budget** : Le budget total de 450 000 € a été respecté à hauteur de 98 %. [PDF](#)

Recommandations

Prompt 2 :

Tu es un traducteur professionnel spécialisé dans les documents d'entreprise.

Tâche : traduis le document suivant du français vers l'anglais.

Contraintes :

- Conserve fidèlement le sens du texte original.
- Conserve la structure du document (paragraphes, listes, titres).
- Conserve les termes techniques tels quels ou utilise leur équivalent standard reconnu dans le domaine.
- Ne résume pas le texte : traduis-le intégralement.
- N'ajoute aucune information, explication ou commentaire absent de l'original.

Document : « Doc_traduction.pdf »

Infrastructure Specifications: Failover Cluster Deployment PDF

The target architecture relies on a failover cluster composed of three active nodes and a file share witness. Data replication between SAN (Storage Area Network) storage arrays is performed synchronously via a dedicated 32 Gbps Fibre Channel network. PDF + 1

In the event of connectivity loss on the primary node, the quorum mechanism initiates an automatic transition of the virtual machines to the secondary node in less than 15 seconds. Load balancing is managed at the network routing layer level by a Next-Generation Firewall (NGFW) configured for high availability. No service interruption is tolerated for the transactional database during this phase. PDF + 2

Prompt 3 :

Tu es un système de tri automatique de tickets pour un service informatique.

Tâche : classe le ticket suivant dans l'une des catégories : réseau, logiciel, matériel, sécurité, accès, autre.

Contraintes :

- Base ta décision uniquement sur le contenu du ticket.
- Si le ticket évoque plusieurs catégories, choisis celle qui correspond au problème principal.
- N'invente aucune information absente du ticket.

Format de sortie : réponds uniquement avec un JSON valide contenant exactement les champs "categorie" et "justification", sans texte additionnel.

Ticket : « Ticket.pdf »

```
{"categorie": "accès", "justification": "Bien que le ticket mentionne des symptômes visuels et une fermeture de l'application, le problème racine partagé par plusieurs utilisateurs réside dans l'impossibilité de se connecter et d'accéder à l'application Sage (erreur 403 : Accès Interdit et lien de réinitialisation de mot de passe inopérant)."}

```

Prompt 4 :

Tu es un système d'extraction d'informations à partir de documents comptables.

Tâche : extrait les informations suivantes de la facture ci-dessous : numéro_facture, date, client, montant_ht, tva, montant_ttc.

Contraintes :

- *Retourne uniquement un JSON valide contenant exactement ces 6 champs.*
- *Si une information est absente ou illisible dans la facture, indique la valeur null pour ce champ, sans l'inventer.*
- *N'ajoute aucun champ supplémentaire, aucun texte avant ou après le JSON.*

Facture : «Facture.pdf»

```
{
  "numéro_facture": "FAC-2026-0899",
  "date": "12 septembre 2026",
  "client": "Cabinet Juridique Durand & Associés",
  "montant_ht": "3 700,00 €",
  "tva": null,
  "montant_ttc": "3 700,00 €"
}
```

Prompt 5 :

Tu es un chargé de relation client pour une entreprise de e-commerce.

Tâche : rédige un email destiné à un client dont la commande a pris du retard de livraison.

Objectifs de l'email :

- *Reconnaître clairement le retard.*
- *Présenter des excuses sincères.*
- *Expliquer la situation sans inventer de cause précise si elle n'est pas connue (rester générique et honnête si besoin).*
- *Proposer une solution ou un geste commercial (ex : suivi, dédommagement, nouvelle estimation de livraison).*

Ton : professionnel, courtois et rassurant.

Contrainte de longueur : maximum 150 mots.

Contexte : « *contexte.pdf* »

The screenshot shows a Gmail compose window. At the top, the Gmail logo is visible. Below it, the 'To' field contains 'novatech@example.com' with a close button (X). To the right of the 'To' field are 'Cc' and 'Bcc' labels. The subject line is 'Suivi et retard de delivery pour votre commande #NT-7741'. The body of the email starts with 'Bonjour,' followed by a paragraph: 'Nous revenons vers vous au sujet de votre commande #NT-7741 concernant les 50 routeurs professionnels [cite: 5].'. Below this, another line of text is partially visible: 'La livraison initialement prévue le 11 septembre subit un retard en raison d'un'. At the bottom of the compose window, there are three buttons: 'Modifier dans Gmail' (with a pencil icon), 'Annuler' (in a light blue button), and 'Envoyer' (in a dark blue button).

Partie 6 — Prompt Engineering pour le Machine Learning

Consigne

Sur le dataset des capteurs :

- (1) stratégies de traitement des valeurs manquantes/doublons/aberrantes/catégorielles,

Rôle : Tu es un ingénieur expert en analyse de données IoT et en gestion technique des bâtiments (Smart Buildings).

Contexte : On te fournit un jeu de données au format CSV contenant les relevés de plusieurs capteurs environnementaux installés dans différents bâtiments d'un site, pour le mois de janvier 2026.

Dictionnaire des colonnes :

- id_mesure : identifiant unique du relevé
- date_heure : date et heure de la mesure (AAAA-MM-JJ HH:MM:SS)
- id_capteur : identifiant unique du capteur
- batiment : identifiant du bâtiment (ex: B001, B002...)

- temperature : température en °C
- humidite : taux d'humidité relative en %
- pression : pression atmosphérique en hPa
- consommation : consommation énergétique mesurée à cet instant
- etat : statut du relevé (OK, ALERTE, ERREUR, ou valeur manquante)

Tâche : propose une stratégie de traitement pour chacun des problèmes suivants, susceptibles d'apparaître dans ce dataset : valeurs manquantes, doublons, valeurs aberrantes, variables catégorielles.

Pour chaque problème, structure ta réponse en 3 points :

- Détection : comment l'identifier, en t'appuyant sur les colonnes concrètes du dataset (ex : etat vide, id_mesure dupliqué, humidite > 100).
- Traitement : la ou les méthodes recommandées.
- Risques associés : ce qu'on risque si ce traitement est mal fait ou omis.

Contraintes : base-toi uniquement sur les colonnes fournies dans le dictionnaire ; ne suppose pas l'existence d'autres variables ; reste spécifique au contexte de capteurs IoT/bâtiments plutôt que de donner des conseils génériques.

Voici les données à analyser : « mesures_capteurs.csv »

1. Valeurs manquantes

- **Détection :**
Identifiez l'absence de données (NaN , null ou chaînes vides) en balayant le jeu de données par colonne. La détection s'effectue notamment sur les mesures physiques (temperature , humidite , pression , consommation) et la variable statutaire (etat). Sur une série temporelle par id_capteur ordonnée par date_heure , repérez également les sauts temporels (ex: absence complète de relevé pour un pas horaire donné).
- **Traitement :**
 - **Métriques environnementales (temperature , humidite , pression) :**
Appliquez une interpolation temporelle (linéaire ou par spline) basée sur les relevés chronologiques (date_heure) du même id_capteur . En cas d'absence courte (1 à 2 pas de temps), le report de la dernière valeur valide (Forward Fill) est privilégié.
 - **Consommation énergétique (consommation) :** Imputez par la valeur médiane calculée sur le même créneau horaire de la semaine et pour le même batiment , ou

(2) visualisations pertinentes pour la consommation énergétique,

Rôle : Tu es un ingénieur expert en analyse de données IoT et en gestion technique des bâtiments (Smart Buildings).

Contexte : On te fournit un jeu de données au format CSV contenant les relevés de plusieurs capteurs environnementaux installés dans différents bâtiments d'un site, pour le mois de janvier 2026.

Dictionnaire des colonnes :

- *id_mesure* : identifiant unique du relevé
- *date_heure* : date et heure de la mesure (AAAA-MM-JJ HH:MM:SS)
- *id_capteur* : identifiant unique du capteur
- *batiment* : identifiant du bâtiment (ex: B001, B002...)
- *temperature* : température en °C
- *humidite* : taux d'humidité relative en %
- *pression* : pression atmosphérique en hPa
- *consommation* : consommation énergétique mesurée à cet instant
- *etat* : statut du relevé (OK, ALERTE, ERREUR, ou valeur manquante)

Tâche : propose les visualisations les plus pertinentes pour comprendre la consommation énergétique des bâtiments à partir de ces données.

Pour chaque visualisation, indique :

- Le type de graphique (courbe temporelle, boxplot, heatmap de corrélation, etc.).
- Les variables utilisées (parmi : *date_heure*, *batiment*, *temperature*, *humidite*, *pression*, *consommation*, *etat*).
- L'objectif de cette visualisation.
- L'interprétation attendue.

Contraintes : limite-toi à 5 visualisations maximum, classées par ordre de priorité d'analyse ; base-toi uniquement sur les colonnes du dictionnaire fourni.

Voici les données à analyser :

1. Profil de consommation temporelle par bâtiment

- **Type de graphique :** Courbe temporelle (Line Plot).
- **Variables utilisées :** `date_heure` , `consommation` , `batiment` .
- **Objectif :** Suivre la dynamique temporelle des consommations énergétiques tout au long du mois de janvier 2026 et comparer le comportement au fil du temps entre les différents bâtiments (`B001` , `B002` , `B003` , `B004`).
- **Interprétation attendue :**
 - Identification de la saisonnalité journalière (cycles heures creuses / heures de pointe) et hebdomadaire (diminution des charges le week-end).
 - Détection immédiate des pics d'anomalies ponctuels (ex. le pic extrême survenu le 22 janvier dans le bâtiment `B003`).

2. Comparaison des distributions énergétiques entre bâtiments

(3) modèles adaptés à la prédiction de consommation énergétique,

Rôle : Tu es un ingénieur expert en analyse de données IoT et en gestion technique des bâtiments (Smart Buildings).

Contexte : On te fournit un jeu de données au format CSV contenant les relevés de plusieurs capteurs environnementaux installés dans différents bâtiments d'un site, pour le mois de janvier 2026.

Dictionnaire des colonnes :

- `id_mesure` : identifiant unique du relevé
- `date_heure` : date et heure de la mesure (AAAA-MM-JJ HH:MM:SS)
- `id_capteur` : identifiant unique du capteur
- `batiment` : identifiant du bâtiment (ex: B001, B002...)
- `temperature` : température en °C
- `humidite` : taux d'humidité relative en %
- `pression` : pression atmosphérique en hPa
- `consommation` : consommation énergétique mesurée à cet instant
- `etat` : statut du relevé (OK, ALERTE, ERREUR, ou valeur manquante)

Tâche : propose 3 à 4 modèles de machine learning adaptés à la prédiction de la variable consommation, à partir des autres colonnes disponibles (temperature, humidite, pression, batiment, date_heure).

Pour chaque modèle, indique :

- Le principe de fonctionnement.
- Les avantages dans ce contexte (données temporelles, multi-bâtiments, multi-capteurs).
- Les limites ou précautions à prendre.
- Le type de problème concerné (régression, série temporelle...).
- Les métriques pertinentes pour évaluer ce modèle.

Contraintes : classe les modèles du plus simple/interprétable au plus complexe ; ne t'appuie que sur les colonnes du dictionnaire fourni.

Voici les données à analyser :

Voici une sélection de 4 modèles de machine learning adaptés à la prédiction de la consommation énergétique, ordonnés du plus simple et interprétable au plus complexe.

1. Régression Linéaire Multiple (avec régularisation Ridge/Lasso)

- **Type de problème** : Régression supervisée (modèle baseline explicatif).
- **Principe de fonctionnement** : Établit une relation linéaire entre la variable cible (consommation) et les variables explicatives directes (temperature , humidite , pression) combinées aux caractéristiques catégorielles encodées (batiment , id_capteur) et aux composantes temporelles extraites de date_heure (ex: heure du jour, jour de la semaine).
- **Avantages dans ce contexte** :
 - **Interprétabilité maximale** : Permet de quantifier directement l'impact de chaque variable (ex: variation de consommation pour chaque $+1^{\circ}\text{C}$ de température).

(4) explication des métriques de classification,

Rôle : Tu es un ingénieur expert en analyse de données IoT, chargé de former une équipe technique à l'évaluation de modèles.

Contexte : dans le cadre du suivi des capteurs de bâtiments, on envisage un modèle de classification permettant de prédire directement la colonne etat (OK, ALERTE, ERREUR) à partir des autres mesures (température, humidité, pression, consommation).

Tâche : explique les métriques de classification suivantes, applicables à cette tâche : Accuracy, Precision, Recall, F1-score, ROC-AUC.

Pour chaque métrique, structure ta réponse en 4 points :

- Définition (formule si pertinent).
- Interprétation concrète.
- Exemple concret, appliqué à la prédiction de l'état d'un capteur (ex : détecter les ALERTE parmi les relevés).
- Contexte d'utilisation privilégié (ex : classes déséquilibrées, comme le fait qu'il y ait probablement beaucoup plus de "OK" que d'"ERREUR").

Contrainte : garde un fil conducteur unique — la détection de l'état ALERTE/ERREUR d'un capteur — pour illustrer toutes les métriques.

Voici le guide de formation sur l'évaluation des métriques de classification pour le suivi fonctionnel des capteurs IoT dans la Gestion Technique des Bâtiments (GTB).

Fil conducteur métier

Dans notre parc IoT, la majorité des relevés sont dans l'état **OK** (fonctionnement nominal), tandis que les états **ALERTE** et **ERREUR** sont minoritaires (classes déséquilibrées). Pour simplifier les illustrations ci-dessous, nous considérons le problème sous l'angle de la détection d'anomalies où :

- **Positif (P)** : Le capteur est en état **ALERTE / ERREUR** (anomalie à détecter).
- **Négatif (N)** : Le capteur est en état **OK** (fonctionnement normal).
- **Vrais Positifs (VP)** : Capteur réellement en ALERTE, correctement identifié.
- **Faux Positifs (FP)** : Capteur normal (OK) signalé à tort en ALERTE (fausse alarme).
- **Vrais Négatifs (VN)** : Capteur normal (OK), correctement identifié comme normal.

(5) explication des métriques de régression.

Rôle : Tu es un ingénieur expert en analyse de données IoT, chargé de former une équipe technique à l'évaluation de modèles.

Contexte : dans le cadre du suivi énergétique des bâtiments, on envisage un modèle de régression permettant de prédire la colonne `consommation` à partir des autres mesures (température, humidité, pression, bâtiment, date/heure).

Tâche : explique les métriques de régression suivantes : MAE, MSE, RMSE.

Pour chaque métrique, structure ta réponse en 4 points :

- *Définition (formule si pertinent).*
- *Interprétation concrète, notamment par rapport à l'unité de la consommation énergétique.*
- *Exemple concret, appliqué à la prédiction de consommation.*
- *Contexte d'utilisation privilégié (ex : sensibilité aux valeurs aberrantes de consommation).*

Contrainte : explique clairement la différence entre MSE et RMSE et pourquoi RMSE est plus facilement interprétable pour un ingénieur suivant la consommation énergétique.

Voici le guide de formation sur l'évaluation des métriques de régression pour la prédiction de la consommation énergétique dans le cadre de la Gestion Technique des Bâtiments (GTB).

Contexte et Notations

Dans le cadre de la prédiction de la consommation énergétique :

- y_i : Consommation réelle mesurée pour le relevé i (ex. en kWh ou kW).
- $\hat{y}_i$: Consommation prédite par le modèle pour le relevé i .
- $e_i = y_i - \hat{y}_i$: Erreur de prédiction (écart entre la réalité et l'estimation).
- N : Nombre total de relevés analysés.

1. MAE (Mean Absolute Error — Erreur Absolue Moyenne)

- **Définition :**

Éléments attendus

- Les 5 prompts rédigés
- Les résultats / réponses obtenues

Partie 7 — Prompt Engineering et RAG

Consigne

À partir d'un document choisi, tester 3 prompts pour une même question : (A) sans fournir le document, (B) en fournissant le document, (C) en fournissant le document avec contraintes anti-hallucination et citation de source. Comparer les résultats.

Éléments attendus

- Document utilisé (référence)
- Résultats des prompts A, B et C
- Comparaison / conclusion

Prompt(s) et résultat(s)

Quelle est la structure de dossier demandée dans l'atelier "Technos Web pour IA" ?

Quelle est la structure de dossier demandée dans l'atelier "Technos Web pour IA" ?

Afficher le code <>

Il n'y a aucun document, fichier ou extrait fourni dans cette conversation qui mentionne l'atelier « **Technos Web pour IA** » ou la structure de dossier demandée pour celui-ci.

Si vous disposez d'un énoncé, d'un sujet de TP/atelier ou d'un document récapitulatif décrivant cet atelier, n'hésitez pas à le partager afin de préciser la structure de projet attendue.

Voici un document :

Quelle est la structure de dossier demandée dans cet atelier ?

D'après le document fourni (Partie 0 - Mise en place de l'environnement), la structure de dossier demandée pour le projet est la suivante : PDF

```
AI-Workspace/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── script.js  
└── images/
```

Explications de l'organisation :

- o AI-Workspace/ : Dossier racine du projet. PDF

Voici un document :

Réponds à la question suivante en utilisant uniquement le contexte fourni ci-dessus, sans t'appuyer sur d'autres connaissances.

Question : Quelle est la structure de dossier demandée dans cet atelier ?

Contraintes :

- N'invente aucune information absente du document.
- Si l'information n'est pas présente dans le document, indique-le explicitement plutôt que de deviner.
- Cite le passage exact du document sur lequel tu bases ta réponse.



Analyse / remarques

Prompt A montre que, même sans contrainte explicite, un modèle peut adopter un comportement prudent face à l'absence de contexte : ici, il a choisi de refuser plutôt que d'halluciner une structure de dossier plausible mais fautive. C'est un résultat positif, mais **non garanti** : rien dans le prompt n'imposait ce comportement, donc il dépend uniquement du modèle utilisé et n'est pas fiable à 100 % dans tous les cas (un autre modèle, ou une question légèrement différente, aurait pu produire une réponse inventée).

Prompt B, une fois le document fourni, donne une réponse exacte, mais le modèle **dépasse le cadre de la question** en ajoutant des explications sur le rôle de chaque fichier cohérentes ici, mais non vérifiables et non demandées. Sans contrainte, on ne peut pas distinguer ce qui vient strictement du document de ce que le modèle a reformulé ou déduit.

Prompt C est le seul des trois à combiner **exactitude et traçabilité** : la réponse est non seulement correcte, mais **prouvée** par une citation exacte du document. C'est la seule version où on peut vérifier objectivement, sans effort supplémentaire, que rien n'a été inventé.

Partie 8 — Évaluation et optimisation des prompts

Consigne

Créer 3 versions de prompt de résumé : (A) « Résume ce texte. », (B) « Résume ce texte en 150 mots », (C) prompt enrichi avec davantage de composants. Évaluer les réponses obtenues.

Éléments attendus

- Résultats des prompts A, B et C
- Grille / critères d'évaluation utilisés
- Conclusion sur la meilleure version

Prompt(s)

Prompt A

Résume ce texte.

Prompt B

Résume ce texte en 150 mots.

Prompt C

Tu es un assistant chargé de produire des résumés exploitables pour des collaborateurs pressés.

Tâche : résume le texte suivant.

Contraintes :

- *Maximum 150 mots.*
- *Conserve les informations factuelles importantes (chiffres, dates, noms propres).*
- *Structure le résumé en identifiant : l'objectif du texte, les points clés, et une conclusion ou recommandation si le texte en contient.*
- *N'invente aucune information absente du texte.*

Format de sortie : réponds avec 3 sections intitulées "Objectif", "Points clés", "Conclusion".

Analyse / remarques

- Le Prompt A donne un résumé **libre**, de longueur imprévisible, sans garantie de structure — bon pour un usage rapide, mauvais pour l'intégration dans une application.
- Le Prompt B ajoute une contrainte de longueur, ce qui améliore la **prévisibilité**, mais ne garantit toujours pas la structure ni la fidélité factuelle.
- Le Prompt C, avec rôle, contraintes explicites et format de sortie imposé, est le plus **contrôlable et exploitable automatiquement** — au prix d'un prompt plus long à écrire. C'est le même constat que pour la Partie 2 : plus de composants de prompt = plus de contrôle sur le résultat, particulièrement utile dans un contexte métier.

